

IT UNIVERSITY OF COPENHAGEN

Machine Learning project

BSMALEA1KU

Fall 2024

Dimitar Kochev
diko@itu.dk

Mihael Stoyanov
svst@itu.dk

Table of Contents

1	Introduction	3
1.1	Exploratory data analysis	3
1.2	Visualization of data	3
2	Details on Machine Learning Methods	4
2.1	Decision Tree	4
2.2	Feed-forward Neural Network	5
2.3	Convolutional Neural Network/CNN/	6
3	Interpretation and discussion of the results	6
3.1	Testing final models	6
3.2	Results	8

Abstract

The paper presents a project that compares three well-known Machine Learning methods. Each method is tasked with classifying a piece of clothing by analyzing a gray-scale image. There are two basic manual implementations of the methods mentioned and three "referenced implementations" that use the resources of specialized libraries like Scikit-learn and TensorFlow are provided. A brief explanation of each model's configuration and interpretation of their respectful performance on a separate test set is provided.

1 Introduction

1.1 Exploratory data analysis

Our data comprises 10,000 train and 5,000 test images of clothing from (Xiao et al., 2017). In it, every picture belongs to one of five classes: T-shirt/top, Trousers, Pullover, Dress, and Shirt. The original images are 28x28 pixels in gray-scale saved as 784 features. We can compare the presence of each class in our training set with the following table:

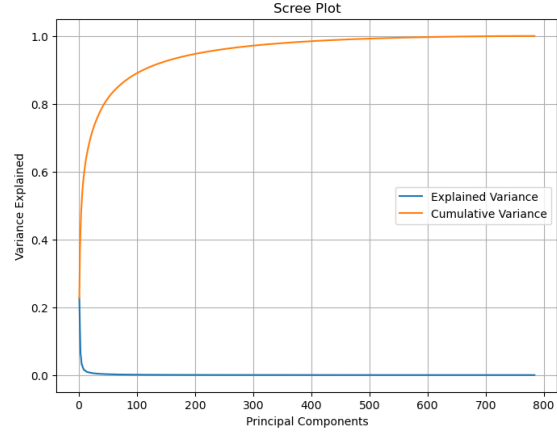
Clothing	T-Shirt/top	Trouser	Pullover	Dress	Shirt
Amount	2033	1947	2001	2005	2014
%	19.5%	20.3%	20.1%	20.0%	20.0%

Class distribution

The table shows an almost equal distribution of the classes. This means that we do not expect to apply any methods for balancing, oversampling, or weighting on classes.

1.2 Visualization of data

In an attempt to gather more information for our data before applying complex machine learning methods, we decided to utilize principal components analysis (PCA). We transform all of our training data and get the following Scree plot:



The idea of PCA is to project our original data into a lower dimension using vectors known also as principal components. The full combination of all components is when we have the normal 784 features which also equals 100% cumulative proportion of explained variance. But if we can reduce the number of features without losing a lot of cumulative variances, we can speed up the learning process without heavily lowering performance. Figure 1 gives an example of how our image can retain its important features despite the lower dimensions:

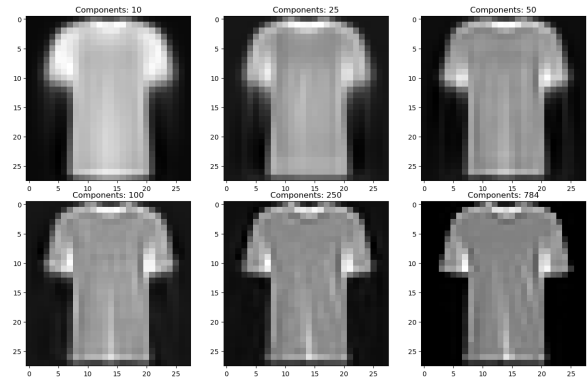


Figure 1. PCA visualization

As one can see, the shape of the cloth is still easy to distinguish even in the reduced dimensions. After further investigation, we decided to test our models with the data explained with 100 components which equals 89.01% of cumulative variances.

2 Details on Machine Learning Methods

2.1 Decision Tree

The decision tree classifier was implemented to classify data from a PCA-reduced feature set based on (Ozeren, 2023). The tree construction process uses entropy as the splitting criterion to maximize information gain at each node. It recursively selects the best feature and threshold to split the data, aiming to reduce entropy and increase homogeneity within the resulting subsets.

The tree-building process begins by calculating the entropy of the dataset, which quantifies the uncertainty of the data for class labels. The algorithm then tests different possible splits across each feature by evaluating various threshold values. For each split, it divides the data into two subsets and computes their respectful weighted entropy. The split that minimizes the total entropy is selected, and this process is repeated recursively for both subsets. The tree stops growing when either the maximum depth is reached or the number of samples in a node falls below the specified threshold (`min_leaf`).

Each node in the tree is represented by a `TreeNode` object, which stores the data at that node, the feature index and threshold used for splitting, and the class probabilities based on the labels in the dataset. The tree structure is again recursive, with each node pointing to its left and right child nodes, continuing until terminal leaf nodes are reached. At the leaf nodes, the model stores the calculated class probabilities, which are then used to predict the class labels for new data points.

For the prediction, the model goes through the whole tree from the root to the appropriate leaf node, based on the feature values of each item. At each decision node, the algorithm compares the feature value to the threshold and proceeds down the left or right branch accordingly. Once the model reaches a leaf node, the class probabilities stored at it are returned, and the class with the highest probability is selected as the final prediction.

The model performance is evaluated using 5-fold cross-validation. The data was divided into five

similarly large subsets, and the model was trained and tested on different combinations of these subsets. The evaluation metric we focus on is accuracy. We also keep track of recall per class, but with the confusion matrix, we can acquire a better sense of how well the model performs across different classes and how well it generalizes to unseen data. Our interpretation is that we want to achieve high accuracy for all data and build a good multi-class classifier but we are also interested in the objects that bring uncertainty to the model. For example, the experiments show a significantly lower performance for pictures from class 'Shirt'.

For implementing the decision tree using integrated libraries, we employed the `sklearn` library. To select the most suitable hyperparameters, Grid Search was applied, as proposed by (Yadav, 2024). Due to the high flexibility of decision trees, they are prone to both overfitting and underfitting.

A tree with excessive depth risks capturing noise as meaningful patterns, leading to overfitting. Conversely, a shallow tree can group unrelated feature values, resulting in underfitting. Additionally, setting a low threshold for the minimum number of items required to form a node may lead to memorizing the training data, ultimately causing poor generalization. These challenges underscore the importance of carefully tuning hyperparameters.

We conducted Grid Search on both the raw data and data reduced by PCA. The process involved evaluating various values for hyperparameters, including the choice between entropy and the Gini coefficient as criteria for split selection. Results indicated that the Gini coefficient performed better on raw data, whereas entropy produced better results on PCA-reduced data. Considering both computation time and the minimal accuracy loss (less than 4%), we opted for the PCA-reduced data. Once the optimal hyperparameters were identified, the same configuration was used for our from-scratch implementation to enable a fair comparison of performance between the two.

2.2 Feed-forward Neural Network

This is a type of artificial neural network (ANN), characterized by the fact that there are no recurrent neurons and all the information is passed through all neurons. Our implementation (model 1.2) of the second required method is based on (Enozeren, 2023). However, to achieve consistent and better results, we have added some strategies for solving general neural network problems like falling in a local minimum and disappearing gradients. After that, we compare it with a similar model which uses the TensorFlow library.

First, we will highlight the differences in the structure design of the basic version presented in the article and our variation. The original relies on building the model using predefined lists containing the number of neurons for each layer. Although the idea is perfect for a functional manual implementation, we felt that this building process was too restricted compared to the methods used in the TensorFlow library. So, to increase building flexibility and model transparency, we altered our original code source to resemble the “reference implementation” which uses an object-oriented approach. Now our implementation allows control over the activation function for each layer, the optimizer, and the learning rate.

Next, we decided to switch to a better optimizer. The original code uses a mini-batch gradient descent but is still susceptible to falling into a local minimum. As we already had He initialization we decided to implement adaptive moment estimation (Adam) based on this article (Kan, 2020). The benefits of Adam are the inclusion of both RMSProp and momentum optimization. The former solves the problem of ending up in a local minimum by introducing the idea of a decaying rate. This means that when we update the weights the most recent gradients have a greater impact on the “direction” of our next step. The momentum is used to accelerate our learning process according to the steepness of the gradient. If the slope is big, there is a chance to overshoot our next step and start oscillating around the minimum. So, the momentum restricts the algorithm from crossing over our target point. However, if the gradient slope is low and there are

no major differences between the updates, taking small steps leads to unnecessary slow learning. Therefore, the momentum can help us cross faster through the steep gradients.

Another important adjustment was the inclusion of Leaky ReLU and batch normalization. That aims to address the disappearing gradient problem that can easily occur during backpropagation due to the imperfections in the derivative of ReLU. Additionally, batch normalization normalizes and scales our data, allowing us to skip these in the preprocessing. Finally, we enabled the use of early stopping with an adjustable patience. This method is pretty helpful during hyperparameter exploration. Specifically for the neural networks it allows us to run training with a high number of epochs, stop it once overfitting is detected, and return to the best state reached beforehand. The algorithms for feed-forward calculations and backpropagation follow the source implementation with the necessary structural adjustments. Having mentioned all the changes in our implementation, we can now focus on our model. After testing different configurations, we saw the best performance in a model containing 5 hidden layers using leaky ReLU and one output layer with a standard SoftMax activation function. The model requires the raw image data and each layer contains half the number of neurons of the previous one. The idea behind it is to have a steady passing of the analysis before we compress it to the final 5 probabilities. So, we begin with an input of 784 features, and pass the weighted sum to 256 neurons, followed by 128 neurons, 64, and 32 on the last hidden layer before our output. Looking at the training curve for the model trained with the full data, after 20 epochs of training it reaches the ceiling of its capacity. In other words, further training does not improve the understanding of our data.

If we try to train the model with our reduced images, we see a big drop in accuracy, meaning that our implementation cannot work well with it. We used the same structure with the raw data for the TensorFlow implementation and the results in accuracy ended up higher with a lower number of epochs required. To our surprise, TensorFlow (model 2.2) does not encounter any issues with the reduced data and performs just as well as the

raw data model. Yet we are going to use the former one because of structure consistency in our comparison.

2.3 Convolutional Neural Network/CNN/

CNN is another type of artificial neural network. Therefore, the method follows tremendously the logic in our previous segment with feed-forward neural networks. However, CNNs have an additional preprocessing part that gives the analysis a great advantage when dealing with special data and the name of the said method. The so-called convolution performs filtering on each image and extracts data for a pixel and its neighbors which leads to developing a better understanding of the shape that defines the object. After that, the preprocessed information is given to a standard network for training. That is why this type of network is infamous for high performance on object recognition tasks. Although the preprocessing aims to apply the same idea of capturing space information, one can implement various tools during the convolution. We will discuss some of them and the reason for their use in our model 2.3.

- **Padding** - This method adds additional space around the image's borders. It usually helps when parts of the object are cut out of the picture. In our case, this is unnecessary because each piece of cloth fits inside the whole image.
- **Stride** - It allows the kernel to skip steps during convolution. This results in a lower amount of data for analysis and faster processing. However, because we use small convolution filters (3x3 pixels), if we apply this method there is a chance that our steps end up being too big and the analysis would not capture precisely the shape of the object. If we had bigger filters, striding would be recommended but then we have to deal with the fact that the output has a greater compression of data which can influence negatively the understanding of details between similar types of clothing.
- **Pooling** - We chose a max-pooling technique since it usually has high performance. We applied 2x2 filters, which is considered a

good strategy when one tries to capture the dominant features of the object.

Once the image goes through the preprocessing, we flatten the output and give it to a fully connected layer for classification. The classification network is similar to our feed-forward implementation with one notable change. We add Dropout on some layers in an attempt to prevent overfitting. Our strategy is to have a high probability for the first hidden layer (0.5), because of the large number of neurons, and decrease it alongside the number of neurons (0.3 on the next one and 0.2 for the last).

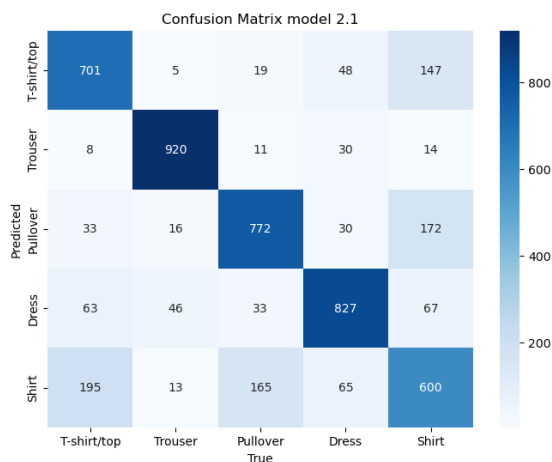
Additionally, we tested the network with L2 regularization, but the cross-validation results were significantly lower so we removed it. The model requires around 15 epochs to reach the best performance. We also decided to further evaluate the performance of our CNN compilation by comparing it to an example from the book (model 2.3.1) (Geron, 2019) which was said to show excellent results in clothes classification. The experiment concludes that the two implementations yield similar accuracy (around 1% difference), but the book implementation takes much longer on average despite the early stopping. Our model proves that it requires less resources to achieve good performance at solving the task, and therefore, it is more worthwhile to use.

3 Interpretation and discussion of the results

3.1 Testing final models

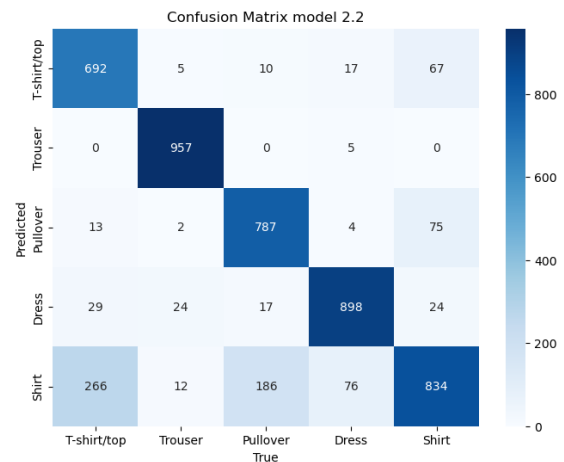
Once we have finished the parameter search for each model, it is time to measure their performance on the test dataset, which contains 5000 additional images that were never shown during training. We mainly focus on test accuracy for evaluation but also provide confusion matrices to spot the most common mistakes during classification and the reason for them.

a) Tree models

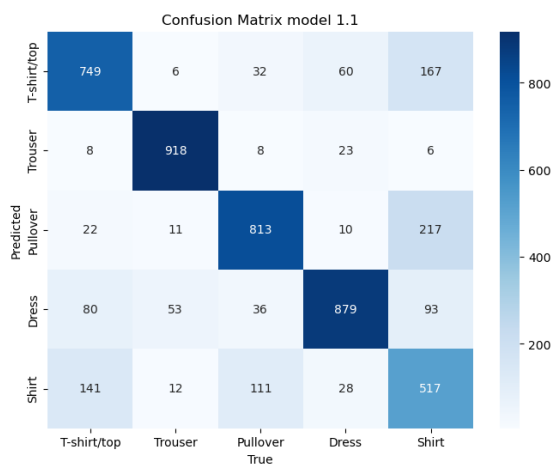


- Decision tree by Scikit-learn (2.1) – our model follows the best result acquired from the grid search with the transformed data. As expected, the solution of the task was too ambitious for the decision tree method and we ended up with an accuracy of around 76.4%. From inspecting the matrix, it is easy to note that the Shirt class is the hardest to identify. The majority of the mistakes are between Shirts and Pullovers or Shirts and T-shirts/tops.

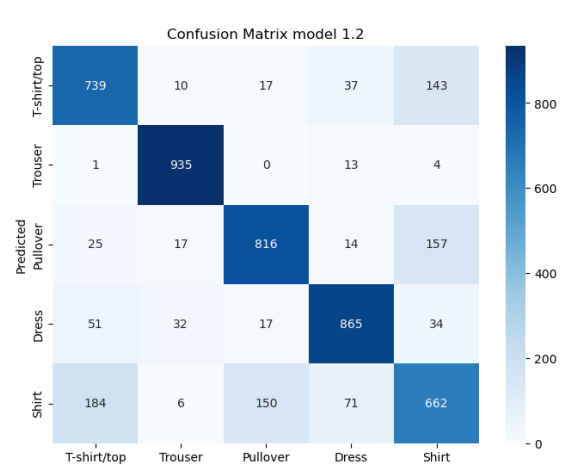
b) Feed-forward neural networks



- Feed-forward network with TensorFlow (2.2) – With 83.4% accuracy, the method easily outperforms the trees. The differentiation between shirts, t-shirts/tops, and pullovers is still the major problem. To our surprise here we see underfitting for the Shirt class. This implies that the model cannot separate it from the other two classes and tries to allocate a big part of the pullovers and T-shirt images to the Shirt class.



- Our Decision tree implementation (1.1) – here we achieved a slightly higher score (77.5%) but the problem with the shirts is yet present and even bigger. However, with the rest of the classes, the model has a consistent if not better score.

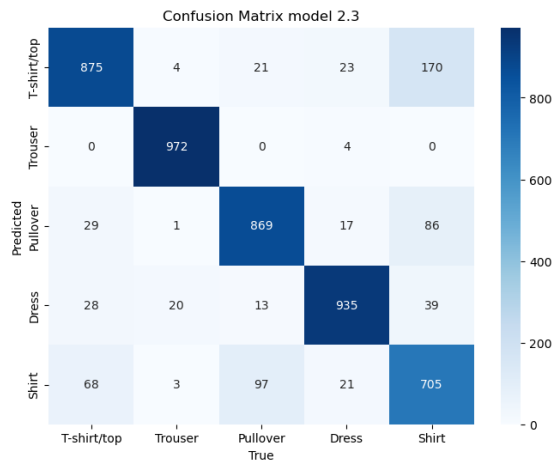


- Our Feed-forward network implementation (1.2) – the accuracy is 80.3%. If we look at the training accuracy curve, we see a fluctuation that is not present in the TensorFlow implementation. Otherwise, the matrix looks more similar to those of the

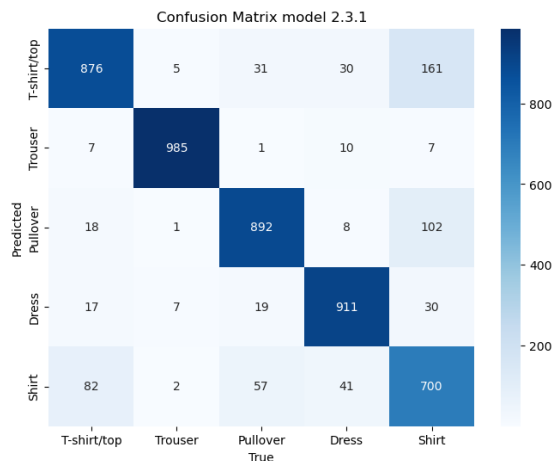
Decision Trees. So there is no extraordinary underfitting in the model.

classification task it will end up too costly.

c) CNN



- Our Convolutional Neural Network with TensorFlow (2.3) – as expected, the method yields a high accuracy of 87.1%. The convolution structure proves to capture well the distinct patterns of each class. Unfortunately, it is not enough to completely solve the Shirt classification issues.



- CNN version from the book (2.3.1) – this model has the highest accuracy of 87.3%, but it is important to point out the fact that it takes a huge amount of time for training. So, as we would like to find a model that compromises on used resources and performance, we might want to discard it. The task in this project is pretty simple, so if we take this model to a more complex

3.2 Results

After our work on the clothing dataset, we reached several important conclusions. First, using PCA proves to be useful enough for simple methods of classification. Second, Decision Trees have a lower performance than ANN when dealing with image classification tasks. Third, CNN models are able to take spatial information into account and detect important patterns that define the object. Without those, the interpretation of our information cannot reach the necessary accuracy for creating a reliable classifier. Finally, we should not blindly focus only on performance and take other factors like computational resources and model simplicity and flexibility on other tasks.

References

- [Enozeren2023] Enozeren. 2023. Building a neural network from scratch with python.
- [Geron2019] Aurelien Geron. 2019. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. O'Reilly Media, Inc., 2nd edition.
- [Kan2020] Enoch Kan. 2020. How to implement an adam optimizer from scratch.
- [Ozeren2023] Enes Ozeren. 2023. Building a decision tree from scratch.
- [Xiao et al.2017] Han Xiao, Kashif Rasul, and Roland Vollgraf. 2017. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. 08.
- [Yadav2024] Amit Yadav. 2024. Grid search for decision tree.